Lab: Ice Cream Store

In the past few labs we have done a deep dive into learning advanced layout methods using CSS. In this lab we will take a break from layout and learn how to add a little 'pizzaz' to a website using CSS, this includes: animations and transitions. We will also discuss the method of using CSS variables and functions to make editing CSS faster and more efficient.

Setup

Download *Lab-Lab-Iceam* and unzip it. It should contain a HTML document called *ice-cream.html*, a CSS document called *style.css* and also includes all of the related images. Load *ice-cream.html* into a code editor.

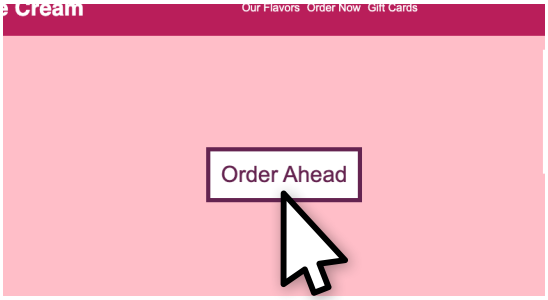


Take a moment to look over the HTML file and CSS file. *<header>*, *<main>* and *<footer>*.

Attached to the HTML document is a stylesheet that is laying out the the basic formatting including typography, colors and basic layouts such as width and heights. You notice that the layout is mostly using CSS grids, with the adveristement in the top right using CSS position.

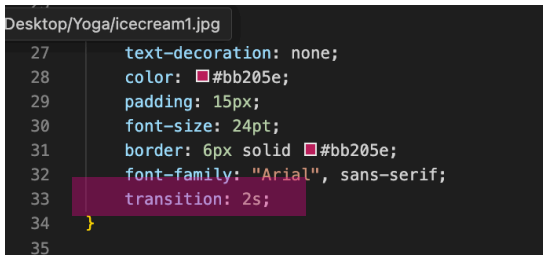
CSS Transitions

Load *ice-cream.html* into a browser. Use your mouse to hover over the *"Order Ahead"* button near the top. You will see that a hover effect has been applied to turn the button from magenta to a dark purple and adds a white background. You'll notice that the switch from magenta to dark purple happens instantaneous. Sometimes for design / animation reasons instead of having the state change happen immediately we would like the state change to ease in or in other words to transition.



CSS transitions allow the style from the normal state to the hover state of an element ease in. In the CSS find the CSS rule called *#splash a* and *#splash a:hover*. You will see that the color and border values change from the normal state or the *#splash a* to *#splash a:hover*. For a transition to occur the CSS property must exist in the normal state and the hover state. In the normal state for the splash area hyperlink add the following:

```
#splash a {
  ...
  transition: color 2s;
}
```



In the value, color is the property to transition, *2s* is how long the transition should take in seconds, in this case two. In CSS we

show time with the measure of *"s"* or seconds. What this tells the browser is transition from the font color *#bb305e* in the normal state to the font color *#682550* over the course of two seconds.

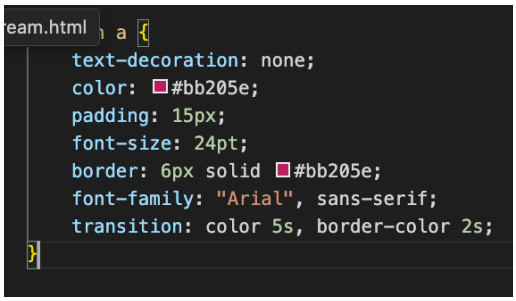
Save and preview and you will notice that instead of the *'jump'* that the switch from font color is gradual. To increase or decrease the speed of the transition change the transition time to more or less seconds. Change the time for *2s* to *5s*. Save and you will see the transition takes longer than the original

Transition Multiple Properties

You'll notice that for the splash area button only the font color transitions, while the other properties such *"jump."* We can fix this issue by setting transitions for multiple properties. Let's animate the border color as well.

Change the CSS code to the following:

```
#splash a {
  ...
  transition: color 5s, border-color 2s;
}
```



This means the color property will take five second to transition and the *border* property will take two seconds. Save and preview, you'll see that the border property transition much faster than the font color.

But, what is you have multiple properties that you want to transition with the same time? If you have multiple properties to transition you can use all to transition all of them in one value. Change the splash hyperlink to:

```
#splash a {
  ...
  transition: all 3s
}
```

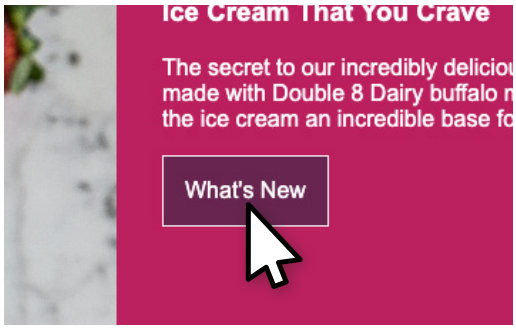




Now all of the properties will transition over three seconds, this includes the font color, border color and background color. Save and preview to view the transition.

Apply Transitions to Other Buttons

Now that you understand the basics let's apply the transition to *#information a*, and *#news-letter a* hyperlinks. Create hovers for *#information a* and *#news-letter a* in the CSS document. Set both to a background-color of *#682550*. Return to *#information a* and create use the transition css property. Make the transitions last three seconds. Repeat the process for *#news-letter a.*



Also lets add a little transition animation to the navigation links and the social media icons in the *<header>* part of the webpage.. Create *:hover* for *nav a* and *#social-media a*. Set the hovers to an off gray of *#b8b8b8*. Make the transitions 1 second.

Transition Movement

Not only can we use transitions to animate font colors and background colors, but we can also use it to make interesting movements. We will use CSS transitions to make it that when we hover over a social media icon that it animates up slightly. To do this will we use a CSS property called transform.

The CSS transform property allows you to manipulate an already existing HTML element. You can use it to move, scale or rotate an element. Let's use it to move a social media icon.

Begin by using the *.social-media* class to set the normal state of the social media icon. Then we will use *transform: translate* to set the offset. Type the following:

```
.social-media {
...
transform: translate(0,0);
}
```

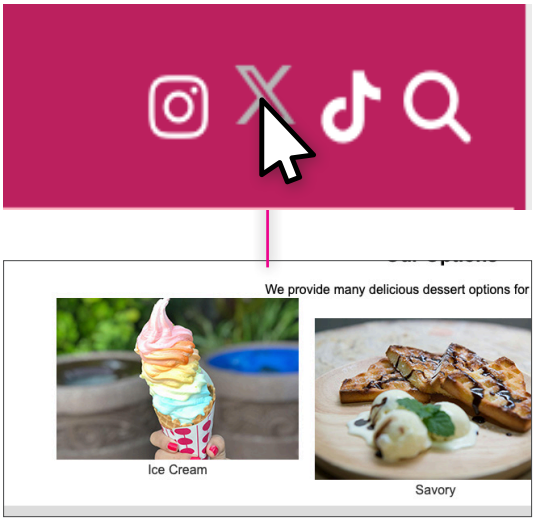


The *translate* value allows you to move or offset an HTML element horizontally and vertically. Setting it to *0* means in the normal state it will remain where it was originally placed. Now let's create a hover state where the the social media icon moves upward slightly. To do this set the *y* value fo translate to a negative number. Type the following:

```
.social-media:hover {
  ....
  transform(0, -5px);
}
```



Save and preview and you'll see that when hovered over the social media icon moves upward. When you remove your mouse it goes back to it's original location. At the moment this is a *'jumpy'*. Let's fix this by adding the CSS transition to the normal state, to do this you will need to set the value of the transition to *all*. Make the transition last 1 second.



Save and preview. You'll see that when you hover the social media icon animates upward and then comes down when the mouse leaves.

Repeat the process for the dessert options. Each dessert option is set using the CSS class *.option*. When the cursor hovers make the option move upward *25 pixels*. Make the transition last one

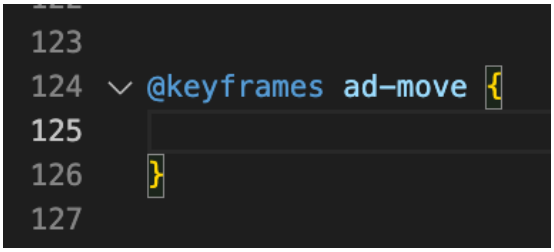
second.

Animation Basics



Cascading Stylesheets has come along way and with every iteration able to control more and more of the design process of a web browser. In the old days animation within the web environment could only be done with browser plugins like Adobe Flash Player or RealPlayer. Now the browser can handle many basic animations using CSS.

The client has a special deal that they want to promote. To help draw attention they want the advertisement to animate from off the top of the page down into the scene to help draw in the user's attention. We can do this with CSS animations and *@keyframes*.



@keyframes allows CSS developers to create an animation. CSS animation has two parts, the animation and the selector that invokes the animation. Let start by creating the animation.

Begin creating the animation. The animation declaration starts with *@keyframes* and must have a name. The name should not have spaces, special characters and the name of the animation is case sensitive. Let's call the animation *ad-move*

```
@keyframes ad-move {
}
```

Now that we declared the animation we need to start what happens during the animation. We want the object that animation is applied to start off the top of the webpage and then come down. We can accomplish this using the CSS top property. Remember with fixed elements the top property allows you to move an HTML element up and down.



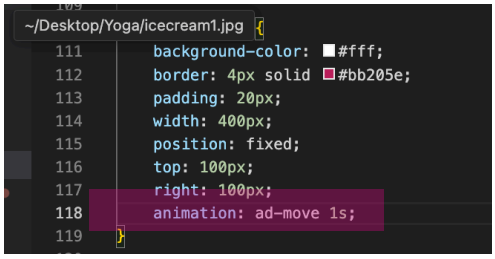
Inside of the brackets use from *{}*. This is what the values of the CSS will be at the beginng. We want the element to start above the webpage, so let's use a negative value, like *-500px*.

```
@keyframes ad-move {
  from { top: -500px}
}
```

Now we need to state where the element will end up at the end of the animation, we'll do this using the to *{}* element. We want it to end at the top value it is right now, which is now at *100px*.

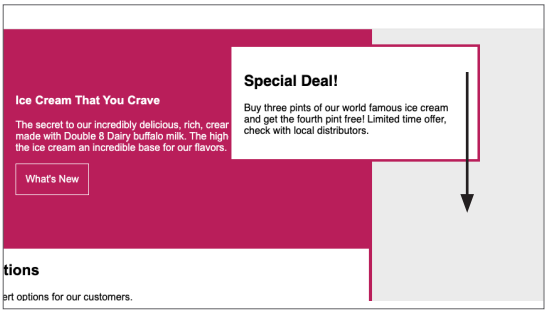
```
@keyframes ad-move {
  from { top: -500px}
     to {top: 100px}
}
```

If you save and preview the webpage you will notice that the ad does not animate, why happened? Once you create an animation with *@keyframes* you need to call it with a selector. The HTML element that is moving is *#advertisement*, so we need to call the animation from there.



Find *#advertisement* in CSS document and add the following:

```
#advertisement {
  ...
  animation: ad-move 1s;
}
```



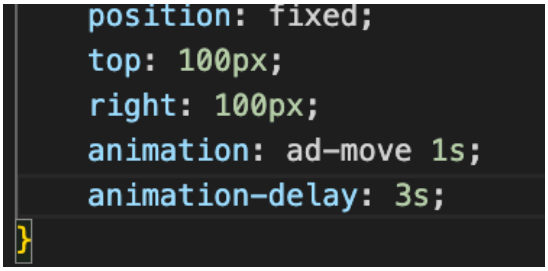
The animation CSS property tells the browser to apply an animation from within the stylesheet. You than call the name of the animation and how long the animation should last, in this case one seconds.

Save and preview and you will see the advertisement animate from the top right of the screen. To replay the animation refresh your webpage. To change the speed of the animation faster or slower change the length of the animation. Change the length from *1s* to *4s*. Save and preview and the animation will go at a slower pace.

Animation Delay

When you load the webpage you will notice that the animation runs immediately. This is because the browser will run the animation as soon as the CSS stylesheet is loaded, which is almost instantanous after the HTML document is loaded into the browser.

But what if you did not want the animation to run immediately? What if we wanted the animation to wait a few seconds before launching? The answer is you can use the CSS property *animation-delay* to have the animation delay before launching. Add the following to the *#advertisement* code in the CSS document.




```
#advertisement {
  ...
  animation: ad-move 4s;
  animation-delay: 3s;
}
```

This tells the browser to delay running the animation for three seconds after the webpage has been loaded.

Save and preview and you'll see that the animation is delayed a few seconds before loading.

Animation State

You may notice that the advertisement exists on the page, then animates to its current location. The issue is since that animation is being delayed 5 seconds it does not immediately start at *-500px* and therefore we can see the advertisement even though it we want it to be off the webpage for the affect of the animation.

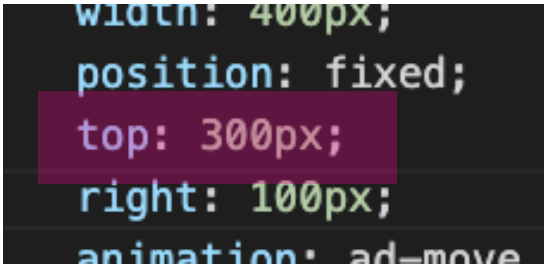
The issue here is where does the element sit before and after the *ad-move* animation we've applied. We would like *#advertisement* to begin at the start value of *-500px* and then stay at the end value of *100px* when the animation is complete. To help with this we can use the CSS property *animation-fill-mode*. The *animation-fill-mode* allows you to have the HTML element begin at the start of the animation, even before the delay has end. It can also allow the HTML element to stay at the end spot of the animation after the animation is complete.

In the *#advertisement* set it to following:

```
#advertisement {
  ...
  animation: ad-move 4s;
  animation-delay: 3s;
  animation-fill-mode: forwards;
}
```

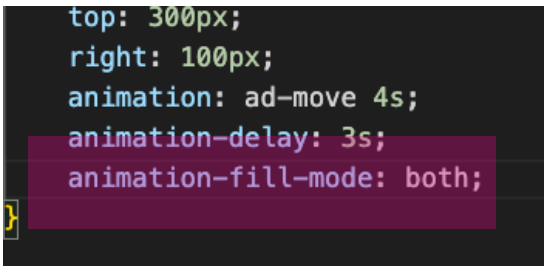


Save and you'll notice that the animation affect appears the same. *#advertisement* stays on screen until the delay is complete than animates. What the *forward* does is set it that the animate will stay at the *to* value of 100px regardless of the original setting.



Find the *#advertisement* and set the top property from *100px* to *300px*. Rerun the animation and when the advertisement come in it will stay at the *100px* end of *ad-move* as opposed to the original of *100px*.

Set the *animation-fill-mode* from *forwards* to *backwards*. Backwards means that the animation will start at the *from* value. Save and run and you will see the advertisement is starts at *-500px*. You will also notice that after the animation ends it jumps back to the original top value of *300px* that we just set, it does not hold the *to* value.



For this animation let's have it start at the from value and then after the animation is done stay at the to value. To do this set the *animation-fill-mode* to both.

Multistep Animation

The client really wants to play up the 'funkiness' of the brand, so they want to create a short animated background behind the *"order here"* button. They wanted the background to animate from *pink* to *blue* to *purple* to *green* and back to *pink* again. This can be done with CSS animations, so let's start.

Begin by creating a new *@keyframes* in the CSS document. Call the animation *bgcolor*.

```
@keyframes bgcolor {
}
```

In the last animation we only had two values for the top property. But for this animation we will need five: *pink*, *blue*, *purple*, *green* and *pink*. How would this work? With *from{}* and *to{}* only gives us two possible values. The answer is there is another way to set a property value point, that is with percentages.

Think for your animation as progress bar that goes from *0* to *100%*. *0%* is the beginning of the animation, and *100%* is the end of the animation. If something happens 1/4th the way through the animation that would *25%*, something that happens about halfway through the animation would be *50%* and so on.

The animated background color starts at pink, so we would write it like this:

```
@keyframes bgcolor {
   0% { background-color: pink}
}
```

About a quarter the way through the animation the color will be *blue*, or *25%*. We would write it like this:

```
@keyframes bgcolor {
    0%  { background-color: pink}
   25%  { background-color: blue}
}
```

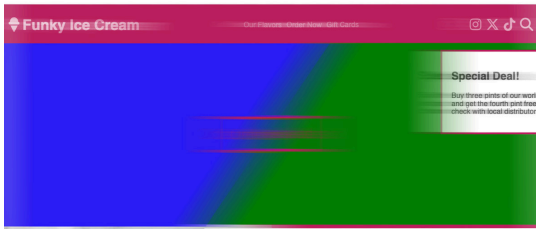
About halfway through the animation the color will be *purple* or *50%*. We would write it like this:

```
@keyframes bgcolor {
    0%  { background-color: pink}
   25% { background-color: blue}
   50% { background-color: purple}
}
```



Add in the remaining two points. At *75%* the background color should be green and at *100%* or the end of the animation the *background-color* should be *pink* again.

Now that we built the animation, let's launch it be calling it from a selector. In the CSS document find *#splash*. Add the *animation* property and call the *bgcolor* animation, have it last for 5 seconds.



Save and preview and you should see that the splash area is now animating between all of the colors. Feel free to add additional colors is you want, just make sure to add in the correct percentage of time.

Looping Animation

One issue you might have noticed is that after the background animation ran, it returned back to a static pink background. CSS animations defaultly only run once, then return back to their default value. If we want the animation to loop more than once we can use the animation-iteraction-count property. In the *#splash* add *animation-iteration-count* and set it *3*.

```
#splash {
 ...
 animation: bgcolor 4s;
 animation-iteration-count:3;
}
```

```
#splash {
    background-color: pink;
    animation: bgcolor 4s;
    animation-iteration-count: 3;
}
```

Save and preview and you will see that the animation runs three times before ending. But what if we want the animation to run indefinately? The answer we can set the value from a number to infinite. infinite means the animation will run forever.

Change the value to *infinite* and save.

Submission

Whew! That was alot, but hopefully you under the basics of CSS transitions and animations.

Please turn all the related lab files to Canvas as a zipped folder for grading.